

CONVERA Command Manual

Overview

This manual describes the public CONVERA command surface. Commands should be run from the project root:

```
cd "/Users/christopherparadise/Documents/New project"
```

Environment Setup

Create and activate a virtual environment:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e .
```

On systems with Python 3.12 installed through Homebrew, this project has also used:

```
/opt/homebrew/bin/python3.12 -m venv .venv312
source .venv312/bin/activate
pip install -e .
```

Health Check

Command:

```
convera health
```

Purpose:

- Verifies that the CLI is available.
- Prints core runtime paths.
- Creates runtime directories if needed.

Expected output shape:

```
[CONVERA] health ok
[PATH] models=...
[PATH] data=...
[PATH] chunks=...
```

Download A Model

Command:

```
python scripts/download_llama.py
```

Default target:

```
meta-llama/Meta-Llama-3-8B
```

Default local path:

```
models/llama3
```

Use a different Hugging Face repo:

```
python scripts/download_llama.py --repo-id "org/model-name"
```

Use a different local directory:

```
python scripts/download_llama.py --local-dir "models/custom-model"
```

Authentication:

```
hf auth login
```

Environment token option:

```
export HF_TOKEN="your-token"
```

Notes:

- Some model repositories require license acceptance in Hugging Face.
- The runtime expects a local model directory before inference.

Run A Prompt

Command:

```
convera run --prompt "Explain neural networks in detail."
```

Optional model path:

```
convera run --model-path "models/llama3" --prompt "Explain neural networks in detail."
```

Optional generation length:

```
convera run --prompt "Explain neural networks in detail." --max-new-tokens 64
```

Optional precision:

```
convera run --precision auto --prompt "Explain neural networks in detail."
convera run --precision fp16 --prompt "Explain neural networks in detail."
convera run --precision int8 --prompt "Explain neural networks in detail."
convera run --precision int4 --prompt "Explain neural networks in detail."
```

Quantized modes require compatible local hardware and dependencies. Unsupported modes fall back safely.

Output includes:

- Generated text.
- Latency.
- Tokens per second.
- KV cache hit or miss.
- Compute avoided percentage.
- Actual precision.
- Validation status when validation mode is enabled or a cache hit is served.
- Backend.
- Memory.

Interactive CLI Mode

Command:

```
convera run
```

Usage:

1. Wait for the CONVERA >> prompt.
2. Type a prompt.
3. Press Enter.
4. Repeat as needed.
5. Type exit or quit to stop.

Encode Model Weights

Command:

```
convera encode
```

Optional model path:

```
convera encode --model-path "models/llama3"
```

Purpose:

- Loads the model.
- Stores tensors through the public CONVERA weight encoding path.
- Reports tensor count and redundancy ratio.

This command uses public-safe fixed-size storage. It does not expose private runtime methods.

Full Benchmark Suite

Command:

```
convera benchmark
```

Equivalent module command:

```
python -m benchmarks.benchmark
```

Optional model path:

```
python -m benchmarks.benchmark --model-path "models/llama3"
```

Optional telemetry:

```
python -m benchmarks.benchmark --send-telemetry
```

Reports:

- Cold run.
- Warm run.
- Variation run.
- Speedup summary.
- Disk growth.
- Generated report path.

Public Repeated-Prompt Benchmark

Command:

```
python -m metrics.benchmark_runner
```

Optional model path:

```
python -m metrics.benchmark_runner --model-path "models/llama3"
```

Optional generation length:

```
python -m metrics.benchmark_runner --max-new-tokens 32
```

Optional precision:

```
python -m metrics.benchmark_runner --precision auto
```

Purpose:

- Runs repeated and slightly varied prompts.
- Reports cache hit status.
- Reports latency in milliseconds.
- Summarizes average latency and repeated-run improvement.
- Saves benchmark JSON and graph images.
- Reports tokens computed, tokens reused, compute avoided percentage, and selected precision.
- Reports validation count and validation overhead when records are generated.

Public-safe output only:

Cached: True or False

Latency: N ms

Generated files:

metrics/output/benchmark.json

metrics/output/benchmark.png

metrics/output/comparison.png

Run The Local UI

Command:

```
uvicorn ui.app:app --reload
```

Open:

<http://127.0.0.1:8000/static/index.html>

Health endpoint:

```
curl http://127.0.0.1:8000/health
```

Metrics endpoint:

```
curl http://127.0.0.1:8000/metrics
```

Status endpoint:

```
curl http://127.0.0.1:8000/status
```

Benchmark graph endpoint:

```
curl http://127.0.0.1:8000/benchmark-graph
```

Run endpoint:

```
curl -X POST http://127.0.0.1:8000/run \  
  -H "Content-Type: application/json" \  
  -d '{"text": "Explain neural networks in detail.", "max_new_tokens": 64, "precision": "auto"}'
```

Validation Mode

Validation mode creates local validation records for route and cache behavior without exposing prompts, outputs, token traces, or private runtime details.

Start the UI with validation mode:

```
CONVERA_VERIFICATION_MODE=1 uvicorn ui.app:app --reload
```

Check status:

```
curl http://127.0.0.1:8000/status
```

Audit Mode

Audit mode creates redacted local traces for each request. Traces include request ID, prompt hash, route mode, precision, latency, token counts, compute avoided percentage, and validation status.

Audit traces do not include prompts, generated outputs, raw tokens, local paths, private formulas, or private runtime internals.

Enable audit mode for every UI run:

```
CONVERA_AUDIT_MODE=1 uvicorn ui.app:app --reload
```

The UI also has an Audit trace checkbox for per-request trace capture.

Latest execution record:

```
curl http://127.0.0.1:8000/api/execution/latest
```

Specific execution record:

```
curl http://127.0.0.1:8000/api/execution/<request_id>
```

Audit trace:

```
curl http://127.0.0.1:8000/audit/<request_id>
```

Export audit trace as JSON:

```
curl http://127.0.0.1:8000/audit/<request_id>/export?format=json
```

Export audit trace as CSV:

```
curl http://127.0.0.1:8000/audit/<request_id>/export?format=csv
```

Telemetry Commands

Telemetry is opt-in. Configure:

```
export CONVERA_METRICS_API_URL="https://your-server.example/api/convera/metrics"  
export CONVERA_METRICS_API_KEY="your-api-key"
```

Enable telemetry for benchmark:

```
python -m benchmarks.benchmark --send-telemetry
```

Privacy:

- No prompts.
- No outputs.
- No local paths.
- No file names.
- No raw tokens.
- No cache contents.

Useful Environment Variables

CONVERA_CHUNK_SIZE	Default tensor chunk size
CONVERA_KV_CHUNK_SIZE	KV tensor chunk size
CONVERA_MAX_NEW_TOKENS	Default generation length
CONVERA_TELEMETRY	Set to 1 for telemetry-aware paths
CONVERA_MODEL_REPO	Model repo for scripts/download_llama.py
HF_TOKEN	Hugging Face access token
CONVERA_METRICS_API_URL	Metrics endpoint
CONVERA_METRICS_API_KEY	Metrics API key

Troubleshooting

Model path does not exist

Run:

```
python scripts/download_llama.py
```

Or provide a local model path:

```
convera run --model-path "models/custom-model" --prompt "Hello"
```

Hugging Face access denied

Check:

- You are logged in with hf auth login.
- You accepted the model license.
- HF_TOKEN is set if running non-interactively.

UI starts but inference fails

Check:

- models/llama3 exists.
- Dependencies are installed.
- The model fits available CPU/GPU memory.

Telemetry does not send

Check:

- CONVERA_METRICS_API_URL is set.
- CONVERA_METRICS_API_KEY is set.
- The server accepts bearer-token auth.